

Python 基础知识清单

变量 · 条件判断 · 循环 · 列表 · 字符串 · 字典 · 简单 NLP 文本处理

本节目标：掌握后续 NumPy、PyTorch 与 NLP 编程最常用的 Python 基础语法，并能独立完成简单的文本分词与词频统计。

复习建议：先看核心语法 → 手敲迷你 NLP Pipeline → 用自测清单检查掌握情况

1. Python 基础：变量与数据类型

输出与变量

```
print("Hello Python")

name = "Tom"
age = 20
score = 3.14
is_student = True
```

Python 变量不需要提前声明类型，可以用 `type()` 查看当前类型。

类型	含义	示例
int	整数	10
float	浮点数	3.14
str	字符串	"hello"
bool	布尔值	True / False

易错点： `=` 表示赋值；`==` 才表示 “是否相等” 。

2. 条件判断：if / elif / else

```
score = 85

if score >= 90:
    print("优秀")
elif score >= 60:
    print("及格")
else:
    print("不及格")
```

Python 从上到下判断条件，一旦某个分支成立，后面的分支不再执行。

运算符	含义	运算符	含义
>	大于	<	小于
>=	大于等于	<=	小于等于
==	等于	!=	不等于

必须记住： Python 用缩进表示代码块，通常使用 4 个空格。

3. 循环：for / while

for 与 range()

```
for i in range(1, 6):  
    print(i)
```

range(start, end, step): 从 start 开始，到 end 之前结束，step 表示步长。

```
range(5)          # 0, 1, 2, 3, 4  
range(1, 6)       # 1, 2, 3, 4, 5  
range(0, 10, 2)   # 0, 2, 4, 6, 8
```

while

```
i = 0  
while i < 5:  
    print(i)  
    i += 1
```

break: 结束整个循环；continue: 跳过本次循环，继续下一次。

易错点：while 循环中要确保条件最终会改变，否则可能出现死循环。

4. 列表 list

```
words = ["I", "love", "NLP"]
```

操作	写法	说明
第一个元素	words[0]	下标从 0 开始
最后一个元素	words[-1]	负数下标从末尾倒数
修改元素	words[1] = "study"	按下标修改
末尾添加	words.append("AI")	新增一个元素
元素个数	len(words)	返回列表长度
判断存在	"NLP" in words	结果为 True / False

遍历列表

```
for word in words:  
    print(word)
```

5. 切片 slicing

通用格式：序列[start:end:step]。其中 end 位置不包含。

```
words[:3] # 前 3 个  
words[2:] # 从下标 2 到末尾  
words[1:4] # 下标 1, 2, 3  
words[::-1] # 反转
```

记忆：切片和 range() 都遵循“左闭右开”——包含开始，不包含结束。

6. 字符串处理

split()：字符串 → 列表

```
sentence = "I love natural language processing"  
words = sentence.split()  
# ['I', 'love', 'natural', 'language', 'processing']
```

join()：列表 → 字符串

```
words = ["I", "love", "NLP"]  
sentence = " ".join(words)  
# I love NLP
```

大小写

```
text.lower() # 全部转小写  
text.upper() # 全部转大写
```

NLP 常见预处理：先 lower() 统一大小写，再 split() 得到 token 列表。

7. 字典 dict

字典保存 key → value 的映射关系。

```
student = {  
    "name": "Tom",  
    "age": 20  
}  
  
print(student["name"])  
student["major"] = "CS"  
student["age"] = 21
```

若 key 不存在，赋值就是添加；若 key 已存在，赋值就是修改。

8. 字典遍历：keys / values / items

方法	作用
dict.keys()	获取所有 key
dict.values()	获取所有 value
dict.items()	同时获取 key 和 value

```
word_count = {  
    "apple": 3,  
    "banana": 2,  
    "orange": 1  
}  
  
for word, count in word_count.items():  
    print(word, count)
```

理解 items()：每次循环会取出一组 (key, value)，分别交给两个变量。

9. 核心应用：词频统计

```
words = ["apple", "banana", "apple", "orange", "banana", "apple"]  
word_count = {}  
  
for word in words:  
    if word in word_count:  
        word_count[word] += 1  
    else:  
        word_count[word] = 1  
  
print(word_count)
```

结果: {'apple': 3, 'banana': 2, 'orange': 1}

逻辑: 第一次遇到单词 → 计数设为 1; 再次遇到 → 原来的计数 +1。

10. 今日迷你 NLP Pipeline

原始文本 → lower() → split() → token 列表 → dict 词频统计

```
sentence = "I Love NLP and I Love Python"
words = sentence.lower().split()

word_count = {}
for word in words:
    if word in word_count:
        word_count[word] += 1
    else:
        word_count[word] = 1

print(words)
print(word_count)
```

后续会继续扩展为: sentence → tokens → token ids → embeddings。

11. 一页自测清单

- 能区分 = 和 ==
- 能写 if / elif / else
- 知道 range() 不包含结束值
- 会使用 for 和 while
- 理解 break / continue
- 知道列表下标从 0 开始
- 会用 append() 和 len()
- 会用切片 [start:end:step]
- 会用 split() 和 join()
- 会用 lower() / upper()
- 理解字典的 key → value
- 会用 items() 遍历键值对
- 能手写简单词频统计
- 能理解基本 NLP 文本预处理流程

下一阶段: tuple、set、列表推导式、dict 推导式、enumerate()、zip()、函数、文件读写, 然后实现第一个 Vocabulary (词表构建器)。